# Chapter 11. PIO

## 11.1. Overview

RP2350 contains 3 identical PIO blocks. Each PIO block has dedicated connections to the bus fabric, GPIO and interrupt controller. The diagram for a single PIO block is shown below in Figure 44.

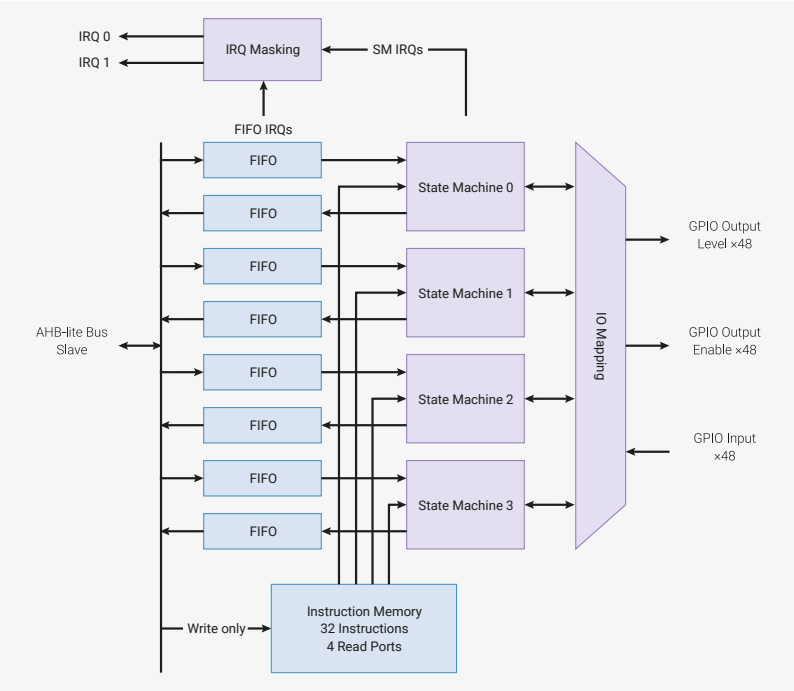


*Figure 44. PIO block-level diagram. There are three PIO blocks, each containing four state machines. The four state machines simultaneously execute programs from shared instruction memory. FIFO data queues buffer data transferred between PIO and the system. GPIO mapping logic allows each state machine to observe and manipulate up to 32 GPIOs.*

The **programmable input/output block** (PIO) is a versatile hardware interface. It can support a variety of IO standards, including:

- 8080 and 6800 parallel bus
- I2C
- 3-pin I2S
- SDIO
- SPI, DSPI, QSPI
- UART
- DPI or VGA (via resistor DAC)

PIO is programmable in the same sense as a processor. There are three PIO blocks with four state machines. Each can independently execute sequential programs to manipulate GPIOs and transfer data. Unlike a general purpose processor, PIO state machines are specialised for IO, with a focus on determinism, precise timing, and close integration with fixed-function hardware. Each state machine is equipped with:

- Two 32-bit shift registers (either direction, any shift count)
- Two 32-bit scratch registers
- 4 × 32-bit bus FIFO in each direction (TX/RX), reconfigurable as 8 × 32 in a single direction
- Fractional clock divider (16 integer, 8 fractional bits)

- Flexible GPIO mapping
- DMA interface (sustained throughput up to 1 word per clock from system DMA)
- IRQ flag set/clear/status

Each state machine, along with its supporting hardware, occupies approximately the same silicon area as a standard serial interface block, such as an SPI or I2C controller. However, PIO state machines can be configured and reconfigured dynamically to implement numerous different interfaces.

Making state machines programmable in a software-like manner, rather than a fully configurable logic fabric like a complex programmable logic device (CPLD), allows more hardware interfaces to be offered in the same cost and power envelope. This also presents a more familiar programming model, and simpler tool flow, to those who wish to exploit PIO's full flexibility by programming it directly, rather than using a pre-made interface from the PIO library.

PIO is performant as well as flexible, thanks to a carefully selected set of fixed-function hardware inside each state machine. When outputting DPI, PIO can sustain 360 Mb/s during the active scanline period when running from a 48 MHz system clock. In this example, one state machine handles frame/scanline timing and generates the pixel clock. Another handles the pixel data and unpacks run-length-encoded scanlines.

State machines' inputs and outputs are mapped to up to 32 GPIOs (limited to 30 GPIOs for RP2350). All state machines have independent, simultaneous access to any GPIO. For example, the standard UART code allows `TX`, `RX`, `CTS` and `RTS` to be any four arbitrary GPIOs, and I2C permits the same for `SDA` and `SCL`. The amount of freedom available depends on how exactly a given PIO program chooses to use PIO's pin mapping resources, but at the minimum, an interface can be freely shifted up or down by some number of GPIOs.

### 11.1.1. Changes from RP2040

RP2350 adds the following new registers and controls:

- DBG_CFGINFO.VERSION indicates the PIO version, to allow PIO feature detection at runtime.
  - This 4-bit field was reserved-0 on RP2040 (indicating version 0), and reads as 1 on RP2350.
- GPIOBASE adds support for more than 32 GPIOs per PIO block.
  - Each PIO block is still limited to 32 GPIOs at a time, but GPIOBASE selects *which* 32.
- CTRL.NEXT_PIO_MASK and CTRL.PREV_PIO_MASK apply some CTRL register operations to state machines in neighbouring PIO blocks simultaneously.
  - CTRL.NEXTPREV_SM_DISABLE stops PIO state machines in multiple PIO blocks simultaneously.
  - CTRL.NEXTPREV_SM_ENABLE starts PIO state machines in multiple PIO blocks simultaneously.
  - CTRL.NEXTPREV_CLKDIV_RESTART synchronises the clock dividers of PIO state machines in multiple PIO blocks
- SM0_SHIFTCTRL.IN_COUNT masks unneeded IN-mapped pins to zero.
  - This is useful for `MOV x, PINS` instructions, which previously always returned a full rotated 32-bit value.
- IRQ0_INTE and IRQ1_INTE now expose all eight SM IRQ flags to system-level interrupts (not just the lower four).
- Registers starting from RXF0_PUTGET0 expose each RX FIFO's internal storage registers for random read *or* write access from the system,
  - The new `FJOIN_RX_PUT` FIFO join mode enables random writes from the state machine, and random reads from the system (for implementing status registers).
  - The new `FJOIN_RX_GET` FIFO join mode enables random reads from the state machine, and random writes from the system (for implementing control registers).
  - Setting both `FJOIN_RX_PUT` and `FJOIN_RX_GET` enables random read *and* write access from the state machine, but disables system access.

RP2350 adds the following new instruction features:

- Adds `PINCTRL_JMP_PIN` as a source for the `WAIT` instruction, plus an offset in the range 0-3.
  - This gives `WAIT` pin arguments a per-SM mapping that is independent of the IN-mapped pins.
- Adds `PINDIRS` as a destination for `MOV`.
  - This allows changing the direction of all OUT-mapped pins with a single instruction: `MOV PINDIRS, NULL` or `MOV PINDIRS, ~NULL`
- Adds SM IRQ flags as a source for `MOV x, STATUS`
  - This allows branching (as well as blocking) on the assertion of SM IRQ flags.
- Extends `IRQ` instruction encoding to allow state machines to set, clear and observe IRQ flags from different PIO blocks.
  - There is no delay penalty for cross-PIO IRQ flags: an `IRQ` on one state machine is observable to all state machines on the next cycle.
- Adds the `FJOIN_RX_GET` FIFO mode.
  - A new `MOV` encoding reads any of the four RX FIFO storage registers into `OSR`.
  - This instruction permits random reads of the four FIFO entries, indexed either by instruction bits or the `Y` scratch register.
- Adds the `FJOIN_RX_PUT` FIFO mode.
  - A new `MOV` encoding writes the `ISR` into any of the four RX FIFO storage registers.
  - The registers are indexed either by instruction bits or the `Y` scratch register.

RP2350 adds the following security features:

- Limits Non-secure PIOs (set to via ACCESSCTRL) to observation of only Non-secure GPIOs. Attempting to read a Secure GPIO returns a 0.
- Disables cross-PIO functionality (IRQs, `CTRL_NEXTPREV` operations) between Non-secure PIO blocks (those which permit Non-secure access according to ACCESSCTRL) and Secure-only blocks (those which do not).

RP2350 includes the following general improvements:

- Increased the number of PIO blocks from two to three (8 → 12 state machines).
- Improved GPIO input/output delay and skew.
- Reduced DMA request (DREQ) latency by one cycle vs RP2040.

## 11.2. Programmer's model

The four state machines execute from shared instruction memory. System software loads programs into this memory, configures the state machines and IO mapping, and then sets the state machines running. PIO programs come from various sources: assembled directly by the user, drawn from the PIO library, or generated programmatically by user software.

From this point on, state machines are generally autonomous, and system software interacts through DMA, interrupts and control registers, as with other peripherals on RP2350. For more complex interfaces, PIO provides a small but flexible set of primitives which allow system software to be more hands-on with state machine control flow.

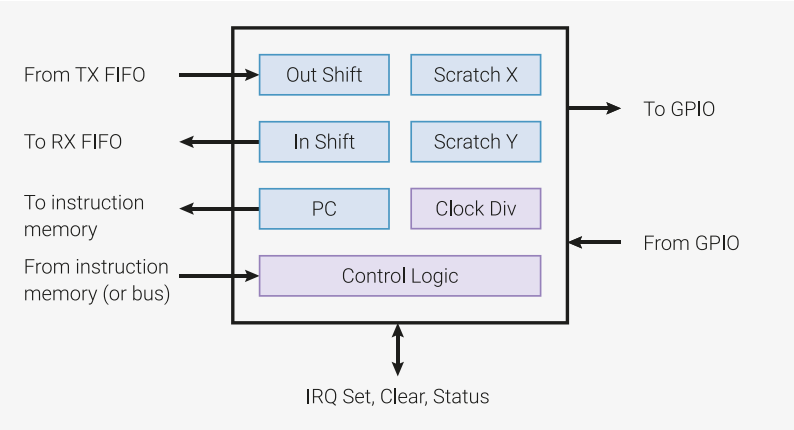


*Figure 45. State machine overview. Data flows in and out through a pair of FIFOs. The state machine executes a program which transfers data between these FIFOs, a set of internal registers, and the pins. The clock divider can reduce the state machine's execution speed by a constant factor.*

## 11.2.1. PIO programs

PIO state machines execute short binary programs.

Programs for common interfaces, such as UART, SPI, or I2C, are available in the PIO library. In many cases, it is not necessary to write PIO programs. However, the PIO is much more flexible when programmed directly, supporting a wide variety of interfaces which may not have been foreseen by its designers.

The PIO has a total of nine instructions: `JMP`, `WAIT`, `IN`, `OUT`, `PUSH`, `PULL`, `MOV`, `IRQ`, and `SET`. For more information about these instructions, see Section 11.4.

Though the PIO only has a total of nine instructions, it would be difficult to edit PIO program binaries by hand. PIO assembly is a textual format, describing a PIO program, where each command corresponds to one instruction in the output binary. The following code snippet contains an example program written in in PIO assembly:

*Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/squarewave/squarewave.pio Lines 8 - 13*

```
 8 .program squarewave
 9     set pindirs, 1   ; Set pin to output
10 again:
11     set pins, 1 [1]  ; Drive pin high and then delay for one cycle
12     set pins, 0      ; Drive pin low
13     jmp again        ; Set PC to label `again`
```

The PIO assembler is included with the SDK, and is called `pioasm`. This program processes a PIO assembly input text file, which may contain multiple programs, and writes out the assembled programs ready for use. For the SDK, these assembled programs are emitted as C headers, containing constant arrays.

For more information, see Section 11.3.

## 11.2.2. Control flow

On every system clock cycle, each state machine fetches, decodes and executes one instruction. Each instruction takes precisely one cycle, unless it explicitly stalls (such as the `WAIT` instruction). Instructions may insert a delay of up to 31 cycles before the next instruction execute, to help write cycle-exact programs.

The program counter, or `PC`, points to the location in the instruction memory being executed on this cycle. Generally, `PC` increments by one each cycle, wrapping at the end of the instruction memory. Jump instructions are an exception and explicitly provide the next value that `PC` will take.

Our example assembly program (listed as `.program squarewave` above) shows both of these concepts in practice. It drives a 50/50 duty cycle square wave with a period of four cycles onto a GPIO. Using some other features (e.g. side-set) this can be made as low as two cycles.

**NOTE**

Side-set is where a state machine drives a small number of GPIOs *in addition to* the main side effects of the instruction it executes. It's described fully in Section 11.5.1.

The system has write-only access to the instruction memory, which is used to load programs. The clock divider slows the state machine's execution by a constant factor, represented as a 16.8 fixed-point fractional number. In the following example, if a clock division of `2.5` were programmed, the square wave would have a period of $4 \times 2.5 = 10$ cycles. This is useful for setting a precise baud rate for a serial interface, such as a UART.

*Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/squarewave/squarewave.c Lines 34 - 38*

```
34    // Load the assembled program directly into the PIO's instruction memory.
35    // Each PIO instance has a 32-slot instruction memory, which all 4 state
36    // machines can see. The system has write-only access.
37    for (uint i = 0; i < count_of(squarewave_program_instructions); ++i)
38        pio->instr_mem[i] = squarewave_program_instructions[i];
```

The following code fragments are part of a complete code example which drives a 12.5 MHz square wave out of GPIO 0 (or any other pins we might choose to map). We can also use pins `WAIT PIN` instruction to stall a state machine's execution for some amount of time, or a `JMP PIN` instruction to branch on the state of a pin, so control flow can vary based on pin state.

*Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/squarewave/squarewave.c Lines 42 - 47*

```
42    // Configure state machine 0 to run at sysclk/2.5. The state machines can
43    // run as fast as one instruction per clock cycle, but we can scale their
44    // speed down uniformly to meet some precise frequency target, e.g. for a
45    // UART baud rate. This register has 16 integer divisor bits and 8
46    // fractional divisor bits.
47    pio->sm[0].clkdiv = (uint32_t) (2.5f * (1 << 16));
```

*Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/squarewave/squarewave.c Lines 51 - 59*

```
51    // There are five pin mapping groups (out, in, set, side-set, jmp pin)
52    // which are used by different instructions or in different circumstances.
53    // Here we're just using SET instructions. Configure state machine 0 SETs
54    // to affect GPIO 0 only; then configure GPIO0 to be controlled by PIO0,
55    // as opposed to e.g. the processors.
56    pio->sm[0].pinctrl =
57            (1 << PIO_SM0_PINCTRL_SET_COUNT_LSB) |
58            (0 << PIO_SM0_PINCTRL_SET_BASE_LSB);
59    gpio_set_function(0, pio_get_funcsel(pio));
```

The system can start and stop each state machine at any time, via the CTRL register. Multiple state machines can be started simultaneously, and the deterministic nature of PIO means they can stay perfectly synchronised.

*Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pio/squarewave/squarewave.c Lines 63 - 67*

```
63    // Set the state machine running. The PIO CTRL register is global within a
64    // PIO instance, so you can start/stop multiple state machines
65    // simultaneously. We're using the register's hardware atomic set alias to
66    // make one bit high without doing a read-modify-write on the register.
67    hw_set_bits(&pio->ctrl, 1 << (PIO_CTRL_SM_ENABLE_LSB + 0));
```

Most instructions are executed from instruction memory, but there are other sources which can be freely mixed:

- Instructions written to a special configuration register (`SMx INSTR`) are immediately executed, momentarily interrupting other execution. For example, a `JMP` instruction written to `SMx INSTR` causes the state machine to start executing from a different location.
- Instructions can be executed from a register, using the `MOV EXEC` instruction.
- Instructions can be executed from the output shifter, using the `OUT EXEC` instruction

The last of these is particularly versatile: instructions can be embedded in the stream of data passing through the FIFO. The I2C example uses this to embed e.g. `STOP` and `RESTART` line conditions alongside normal data. In the case of `MOV` and `OUT EXEC`, the `MOV`/`OUT` itself executes in one cycle, and the executee on the next.

## 11.2.3. Registers

Each state machine possesses a small number of internal registers. These hold input or output data, and temporary values such as loop counter variables.

### 11.2.3.1. Output Shift Register (OSR)

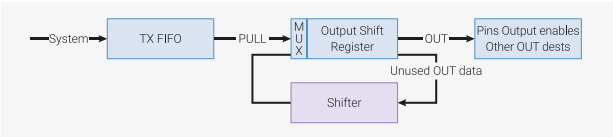


*Figure 46. Output Shift Register (OSR). Data is parcelled out 1…32 bits at a time, and unused data is recycled by a bidirectional shifter. Once empty, the OSR is reloaded from the TX FIFO.*

The Output Shift Register (OSR) holds and shifts output data between the TX FIFO and the pins or other destinations, such as the scratch registers.

- `PULL` instructions: remove a 32-bit word from the TX FIFO and place into the OSR.
- `OUT` instructions shift data from the OSR to other destinations, 1…32 bits at a time.
- The OSR fills with zeroes as data is shifted out
- The state machine will automatically refill the OSR from the FIFO on an `OUT` instruction, once some total shift count threshold is reached, if autopull is enabled
- Shift direction can be left/right, configurable by the processor via configuration registers

For example, to stream data through the FIFO and output to the pins at a rate of one byte per two clocks:

```
.program pull_example1
loop:
    out pins, 8
public entry_point:
    pull
    out pins, 8 [1]
    out pins, 8 [1]
    out pins, 8
    jmp loop
```

## 11.2.4. Autopull

**Autopull** (see Section 11.5.4) allows the hardware to automatically refill the OSR in the majority of cases, with the state machine stalling if it tries to `OUT` from an empty OSR. This has two benefits:

- No instructions spent on explicitly pulling from FIFO at the right time
- Higher throughput: can output up to 32 bits on every single clock cycle, if the FIFO stays topped up

After configuring autopull, the above program can be simplified to the following, which behaves identically:

```
.program pull_example2

loop:
    out pins, 8
public entry_point:
    jmp loop
```

Program wrapping (Section 11.5.2) allows further simplification and, if desired, an output of 1 byte every system clock cycle.

```
.program pull_example3

public entry_point:
.wrap_target
    out pins, 8 [1]
.wrap
```

### 11.2.4.1. Input Shift Register (ISR)

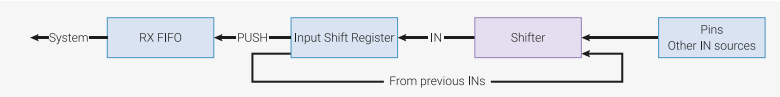


*Figure 47. Input Shift Register (ISR). Data enters 1…32 bits at a time, and current contents is shifted left or right to make room. Once full, contents is written to the RX FIFO.*

- `IN` instructions shift 1…32 bits at a time into the register.
- `PUSH` instructions write the ISR contents to the RX FIFO.
- The ISR is cleared to all-zeroes when pushed.
- The state machine will automatically push the ISR on an `IN` instruction, once some shift threshold is reached, if autopush is enabled.
- Shift direction is configurable by the processor via configuration registers

Some peripherals, like UARTs, must shift in from the left to get correct bit order, since the wire order is LSB-first; however, the processor may expect the resulting byte to be right-aligned. This is solved by the special `null` input source, which allows the programmer to shift some number of zeroes into the ISR, following the data.

### 11.2.4.2. Shift counters

State machines remember how many bits, in total, have been shifted out of the OSR via `OUT` instructions, and into the `ISR` via `IN` instructions. This information is tracked at all times by a pair of hardware counters: the output shift counter and the input shift counter. Each is capable of holding values from 0 to 32 inclusive. With each shift operation, the relevant counter increments by the shift count, up to the maximum value of 32 (equal to the width of the shift register). The state machine can be configured to perform certain actions when a counter reaches a configurable threshold:

- The OSR can be automatically refilled once some number of bits have been shifted out (see Section 11.5.4).
- The ISR can be automatically emptied once some number of bits have been shifted in (see Section 11.5.4.

- `PUSH` or `PULL` instructions can be conditioned on the input or output shift counter, respectively.

On PIO reset, or the assertion of `CTRL_SM_RESTART`, the input shift counter is cleared to 0 (nothing yet shifted in), and the output shift counter is initialised to 32 (nothing remaining to be shifted out; fully exhausted). Some other instructions affect the shift counters:

- A successful `PULL` clears the output shift counter to 0
- A successful `PUSH` clears the input shift counter to 0
- `MOV OSR, ⋯` (i.e. any `MOV` instruction that writes `OSR`) clears the output shift counter to 0
- `MOV ISR, ⋯` (i.e. any `MOV` instruction that writes `ISR`) clears the input shift counter to 0
- `OUT ISR, count` sets the input shift counter to `count`

### 11.2.4.3. Scratch registers

Each state machine has two 32-bit internal scratch registers, called `X` and `Y`.

They are used as:

- Source/destination for `IN`/`OUT`/`SET`/`MOV`
- Source for branch conditions

For example, suppose we wanted to produce a long pulse for "1" data bits, and a short pulse for "0" data bits:

```
.program ws2812_led

public entry_point:
    pull
    set x, 23       ; Loop over 24 bits
bitloop:
    set pins, 1     ; Drive pin high
    out y, 1 [5]    ; Shift 1 bit out, and write it to y
    jmp !y skip     ; Skip the extra delay if the bit was 0
    nop [5]
skip:
    set pins, 0 [5]
    jmp x-- bitloop ; Jump if x nonzero, and decrement x
    jmp entry_point
```

Here `X` is used as a loop counter, and `Y` is used as a temporary variable for branching on single bits from the OSR. This program can be used to drive a WS2812 LED interface, although more compact implementations are possible (as few as 3 instructions).

`MOV` allows the use of the scratch registers to save/restore the shift registers if, for example, you would like to repeatedly shift out the same sequence.

**NOTE**

A much more compact WS2812 example (4 instructions total) is shown in Section 11.6.2.

### 11.2.4.4. FIFOs

Each state machine has a pair of 4-word deep FIFOs, one for data transfer from system to state machine (TX), and the other for state machine to system (RX). The TX FIFO is written to by system bus masters, such as a processor or DMA controller, and the RX FIFO is written to by the state machine. FIFOs decouple the timing of the PIO state machines and the system bus, allowing state machines to go for longer periods without processor intervention.

FIFOs also generate data request (DREQ) signals, which allow a system DMA controller to pace its reads/writes based on the presence of data in an RX FIFO, or space for new data in a TX FIFO. This allows a processor to set up a long transaction, potentially involving many kilobytes of data, which will proceed with no further processor intervention.

Often, a state machine only transfers data in one direction. In this case, the `SHIFTCTRL_FJOIN` option can merge the two FIFOs into a single 8-entry FIFO that only goes in one direction. This is useful for high-bandwidth interfaces such as DPI.

### 11.2.5. Stalling

State machines may momentarily pause execution for a number of reasons:

- A `WAIT` instruction's condition is not yet met
- A blocking `PULL` when the TX FIFO is empty, or a blocking `PUSH` when the RX FIFO is full
- An `IRQ WAIT` instruction which has set an `IRQ` flag, and is waiting for it to clear
- An `OUT` instruction when autopull is enabled, and OSR has already reached its shift threshold
- An `IN` instruction when autopush is enabled, ISR reaches its shift threshold, and the RX FIFO is full

In this case, the program counter does not advance, and the state machine will continue executing this instruction on the next cycle. If the instruction specifies some number of delay cycles before the next instruction starts, these do not begin until *after* the stall clears.

**NOTE**

> Side-set (Section 11.5.1) is not affected by stalls, and always takes place on the first cycle of the attached instruction.

### 11.2.6. Pin mapping

PIO controls the output level and direction of up to 32 GPIOs, and can observe their input levels. On every system clock cycle, each state machine may do none, one, or both of the following:

- Change the level or direction of some GPIOs via an `OUT` or `SET` instruction, or read some GPIOs via an `IN` instruction
- Change the level or direction of some GPIOs via a side-set operation

Each of these operations uses one of four contiguous ranges of GPIOs, with the base and count of each range configured via each state machine's `PINCTRL` register. There is a range for each of `OUT`, `SET`, `IN` and side-set operations. Each range can cover any of the GPIOs accessible to a given PIO block (on RP2350 this is the 30 user GPIOs), and the ranges can overlap.

For each individual GPIO output (level and direction separately), PIO considers all 8 writes that may have occurred on that cycle, and applies the write from the highest-numbered state machine. If the same state machine performs a `SET`/`OUT` and a side-set on the same GPIO simultaneously, the side-set is used. If no state machine writes to this GPIO output, its value does not change from the previous cycle.

Generally each state machine's outputs are mapped to a distinct group of GPIOs, implementing some peripheral interface.

### 11.2.7. IRQ flags

IRQ flags are state bits which can be set or cleared by state machines or the system. There are 8 in total: all 8 are visible to all state machines, and the lower 4 can also be masked into one of PIO's interrupt request lines, via the `IRQ0_INTE` and `IRQ1_INTE` control registers.

They have two main uses:

- Asserting system level interrupts from a state machine program, and optionally waiting for the interrupt to be acknowledged
- Synchronising execution between two state machines

State machines interact with the flags via the `IRQ` and `WAIT` instructions.

### 11.2.8. Interactions between state machines

Instruction memory is implemented as a 1-write, 4-read register file, allowing all four state machines to read an instruction on the same cycle without stalling.

There are three ways to apply the multiple state machines:

- Pointing multiple state machines at the same program
- Pointing multiple state machines at different programs
- Using multiple state machines to run different parts of the same interface, e.g. TX and RX side of a UART, or clock/hsync and pixel data on a DPI display

State machines cannot communicate data, but they can synchronise with one another by using the IRQ flags. There are 8 flags total. Each state machine can set or clear any flag using the `IRQ` instruction, and can wait for a flag to go high or low using the `WAIT IRQ` instruction. This allows cycle-accurate synchronisation between state machines.

## 11.3. PIO assembler (pioasm)

The PIO Assembler parses a PIO source file and outputs the assembled version ready for inclusion in an RP2350 application. This includes C and C++ applications built against the SDK, and Python programs running on the RP2350 MicroPython port.

This section briefly introduces the directives and instructions that can be used in `pioasm` input. For a deeper discussion of how to use `pioasm`, how it is integrated into the SDK build system, extended features such as code pass through, and the various output formats it can produce, see Raspberry Pi Pico-series C/C++ SDK.

### 11.3.1. Directives

The following directives control the assembly of PIO programs:

**`.define (PUBLIC) <symbol> <value>`**

Define an integer symbol named `<symbol>` with the value `<value>` (see Section 11.3.2). If this `.define` appears before the first program in the input file, then this define is global to all programs, otherwise it is local to the program in which it occurs. If `PUBLIC` is specified, the symbol will be emitted into the assembled output for use by user code. For the SDK this takes the following forms:

- `#define <program_name> <symbol> value`: for program symbols
- `#define <symbol> value`: for global symbols

**`.clock_div <divider>`**

If this directive is present, `<divider>` is the state machine clock divider for the program. Note, that divider is a floating point value, but *may not* currently use arithmetic expressions or defined values. This directive affects the default state machine configuration for a program. This directive is only valid within a program before the first instruction.

**`.fifo <fifo_config>`**

If this directive is present, it is used to specify the FIFO configuration for the program. It affects the default state machine configuration for a program, but also restricts what instructions may be used (for example PUSH makes

no sense if there is no IN FIFO configured).

This directive supports the following configuration values:

- `txrx`: 4 FIFO entries for each of TX and RX; this is the default.
- `tx`: All 8 FIFO entries for TX.
- `rx`: All 8 FIFO entries for RX.
- `txput`: 4 FIFO entries for TX, and 4 FIFO entries for `mov rxfifo[index], isr` aka `put`. This value is not supported on PIO version 0.
- `txget`: 4 FIFO entries for TX, and 4 FIFO entries for `mov osr, rxfifo[index]` aka `get`. This value is not supported on PIO version 0.
- `putget`: 4 FIFO entries for `mov rxfifo[index], isr` aka `put`, and 4 FIFO entries for `mov osr, rxfifo[index]` aka `get`. This value is not supported on PIO version 0.

  This directive is only valid within a program before the first instruction.

`.mov_status rxfifo < <n>`

`.mov_status txfifo < <n>`

`.mov_status irq <(prev|next)> set <n>`

This directive configures the source for the `mov , STATUS`. One of the three syntaxes can be used to set the status based on the RXFIFO level being below a value N, the TXFIFO level being below a value N, or an IRQ flag N being set on this PIO instance (or the next lower numbered, or higher numbered PIO instance if `prev` or `next` or specified). Note, that the IRQ option requires PIO version 1.

This directive affects the default state machine configuration for a program. This directive is only valid within a program before the first instruction.

`.in <count> (left|right) (auto) (<threshold>)`

If this directive is present, `<count>` indicates the number of IN bits to be used. 'left' or 'right' if specified, control the ISR shift direction; 'auto', if present, enables "auto-push"; `<threshold>`, if present, specifies the "auto-push" threshold. This directive affects the default state machine configuration for a program.

This directive is only valid within a program before the first instruction. When assembling for PIO version 0, `<count>` must be 32.

`.program <name>`

Start a new program with the name `<name>`. Note that that name is used in code so should be alphanumeric/underscore not starting with a digit. The program lasts until another `.program` directive or the end of the source file. PIO instructions are only allowed within a program.

`.origin <offset>`

Optional directive to specify the PIO instruction memory offset at which the program *must* load. Most commonly this is used for programs that must load at offset 0, because they use data based JMPs with the (absolute) jmp target being stored in only a few bits. This directive is invalid outside a program.

`.out <count> (left|right) (auto) (<threshold>)`

If this directive is present, `<count>` indicates the number of OUT bits to be used. 'left' or 'right' if specified control the OSR shift direction; 'auto', if present, enables "auto-pull"; `<threshold>`, if present, specifies the "auto-pull" threshold. This directive affects the default state machine configuration for a program. This directive is only valid within a program before the first instruction.

`.pio_version <version>`

This directive sets the target PIO hardware version. The version for RP2350 is `1` or `RP2350`, and is also the default version number. For backwards compatibility with RP2040, `0` or `RP2040` may be used.

If this directive appears before the first program in the input file, then this define is the default for all programs, otherwise it specifies the version for the program in which it occurs. If specified for a program, it must occur before the first instruction.

`.set <count>`

If this directive is present, `<count>` indicates the number of SET bits to be used. This directive affects the default state machine configuration for a program. This directive is only valid within a program before the first instruction.

`.side_set <count> (opt) (pindirs)`

If this directive is present, `<count>` indicates the number of side-set bits to be used. Additionally, `opt` may be specified to indicate that a `side <value>` is optional for instructions (note this requires stealing an extra bit — in addition to the `<count>` bits — from those available for the instruction delay). Finally, `pindirs` may be specified to indicate that the side set values should be applied to the PINDIRs and not the PINs. This directive is only valid within a program before the first instruction.

`.wrap_target`

Place prior to an instruction, this directive specifies the instruction where execution continues due to program wrapping. This directive is invalid outside of a program, may only be used once within a program, and if not specified defaults to the start of the program.

`.wrap`

Placed after an instruction, this directive specifies the instruction after which, in normal control flow (i.e. `jmp` with false condition, or no `jmp`), the program wraps (to `.wrap_target` instruction). This directive is invalid outside of a program, may only be used once within a program, and if not specified defaults to after the last program instruction.

`.lang_opt <lang> <name> <option>`

Specifies an option for the program related to a particular language generator. (See Language Generators in Raspberry Pi Pico-series C/C++ SDK). This directive is invalid outside of a program.

`.word <value>`

Stores a raw 16-bit value as an instruction in the program. This directive is invalid outside of a program.

## 11.3.2. Values

The following types of values can be used to define integer numbers or branch targets:

*Table 978. Values in pioasm, i.e. `<value>`*

| | |
|---|---|
| `integer` | An integer value, e.g. 3 or -7. |
| `hex` | A hexadecimal value, e.g. `0xf`. |
| `binary` | A binary value, e.g. `0b1001`. |
| `symbol` | A value defined by a `.define` (see `pioasm_define`). |
| `<label>` | The instruction offset of the label within the program. Typically used with a JMP instruction (see Section 11.4.2). |
| `(<expression>)` | An expression to be evaluated; see expressions. Note that the parentheses are necessary. |

## 11.3.3. Expressions

Expressions may be freely used within pioasm values.

*Table 979. Expressions in pioasm i.e. `<expression>`*

| | |
|---|---|
| `<expression> + <expression>` | The sum of two expressions |
| `<expression> - <expression>` | The difference of two expressions |
| `<expression> * <expression>` | The multiplication of two expressions |
| `<expression> / <expression>` | The integer division of two expressions |
| `- <expression>` | The negation of another expression |

| | |
|---|---|
| `<expression> << <expression>` | One expression shifted left by another expression |
| `<expression> >> <expression>` | One expression shifted right by another expression |
| `:: <expression>` | The bit reverse of another expression |
| `<value>` | Any value (see Section 11.3.2) |

### 11.3.4. Comments

To create a line comment that ignores all content on a certain line following a certain symbol, use `//` or `;`.

To create a C-style block comment that ignores all content across multiple lines until after a start symbol until an end symbol appears, use `/*` to begin the comment and `*/` to end the comment.

### 11.3.5. Labels

Labels use the following forms at the start of a line:

```
<symbol>:
```

```
PUBLIC <symbol>:
```

**TIP**

A label is really just an automatic `.define` with a value set to the current program instruction offset. A `PUBLIC` label is exposed to the user code in the same way as a `PUBLIC .define`.

### 11.3.6. Instructions

All pioasm instructions follow a common pattern:

```
<instruction> (side <side_set_value>) ([<delay_value>])
```

where:

`<instruction>` An assembly instruction detailed in the following sections. (see Section 11.4)

`<side_set_value>` A value (see Section 11.3.2) to apply to the side_set pins at the start of the instruction. Note that the rules for a side-set value via `side <side_set_value>` are dependent on the `.side_set` (see `pioasm_side_set`) directive for the program. If no `.side_set` is specified then the `side <side_set_value>` is invalid, if an optional number of sideset pins is specified then `side <side_set_value>` may be present, and if a non-optional number of sideset pins is specified, then `side <side_set_value>` is required. The `<side_set_value>` must fit within the number of side-set bits specified in the `.side_set` directive.